

AMRITA SCHOOL OF ENGINEERING, BENGALURU AMRITA  
VISHWA VIDYAPEETHAM  
BENGALURU, 560 035

# **Iot System Level Design**

## **Bluetooth Operated RC Car**

Course: Introduction to Internet of Things (23ECE184)  
Department: Electronics & Communication Engineering (ECE)



*Submitted by:-*

BL.EN.U4ECE25123: Lohith  
BL.EN.U4ECE25125: Nishit Agarwal  
BL.EN.U4ECE25126: Sujoth

## 1. Title

# Bluetooth-Controlled RC Car using Arduino Uno and MIT App Inventor

## 2. Objectives

The primary objectives of this project are:

- Design and implement a four-wheeled RC car controllable via Bluetooth from a smartphone application.
- Develop a custom Android application using MIT App Inventor to send directional commands (Forward, Backward, Left, Right, Stop) to the car.
- Implement PWM-based motor speed control through a real-time slider in the app, allowing variable speed between PWM 100 and 180.
- Integrate an L298N dual H-bridge motor driver to control four DC motors independently using Arduino digital and PWM pins.
- Demonstrate bidirectional wireless communication via the HC-05 Bluetooth module operating over UART at 9600 baud.
- Build a robust command-parsing system with a 100 ms timeout watchdog to handle Bluetooth noise and prevent the car from freezing mid-operation.

## 3. Components Required

| Component            | Specification           | Quantity | Purpose                  |
|----------------------|-------------------------|----------|--------------------------|
| Arduino Uno R3       | ATmega328P, 16 MHz      | 1        | Main microcontroller     |
| L298N Motor Driver   | Dual H-bridge, 2A/ch    | 1        | Drive 4 DC motors        |
| HC-05 Bluetooth      | SPP, 2.4 GHz, 9600 baud | 1        | Wireless communication   |
| DC Gear Motors       | 3–6V, with wheels       | 4        | Locomotion (2 per side)  |
| 12V Battery / Pack   | 12V, 1–2 Ah             | 1        | Motor power supply       |
| Capacitors           | 0.1µF                   | 4 sets   | Motor noise suppression  |
| Jumper Wires         | M-M, M-F                | Multiple | Circuit connections      |
| RC Car Chassis       | 4-wheel platform        | 1        | Physical frame           |
| Smartphone (Android) | Android 5.0+            | 1        | Run MIT App Inventor app |

## 4. Block Diagram

The system architecture follows a client-server IoT model. The smartphone acts as the controller (client), transmitting directional and speed commands over Bluetooth. The HC-05 module receives these commands and relays them to the Arduino via UART. The Arduino processes each command and drives the L298N motor driver, which in turn powers the DC motors. A separate regulated 5V supply from the L298N powers the Arduino, isolating it from motor voltage spikes.

Figure 1: Block Diagram - Bluetooth RC Car System

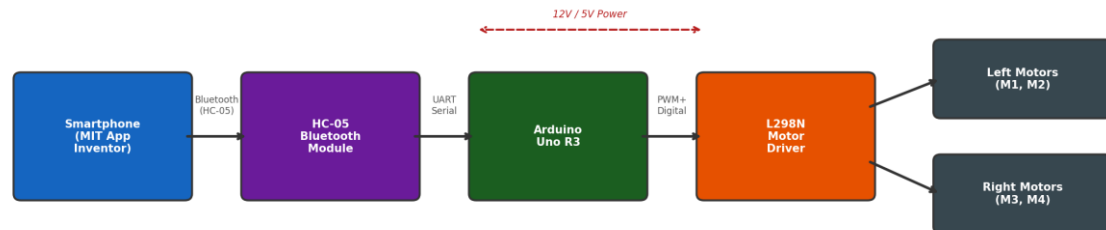


Figure 1: Block Diagram – Bluetooth RC Car System

## 5. Circuit Diagram

### 5.1 Pin Connections

| Arduino Pin  | Connected To            | Signal Type        |
|--------------|-------------------------|--------------------|
| Pin 12 (RX)  | HC-05 TX                | UART Serial        |
| Pin 13 (TX)  | HC-05 RX                | UART Serial        |
| 5V           | HC-05 VCC + L298N 5V in | 5V Power           |
| GND          | HC-05 GND + L298N GND   | Common Ground      |
| Pin 3 (ENA)  | L298N ENA               | PWM Speed (Left)   |
| Pin 5 (ENB)  | L298N ENB               | PWM Speed (Right)  |
| Pin 9 (IN1)  | L298N IN1               | Left Motor Dir A   |
| Pin 10 (IN2) | L298N IN2               | Left Motor Dir B   |
| Pin 11 (IN3) | L298N IN3               | Right Motor Dir A  |
| Pin 6 (IN4)  | L298N IN4               | Right Motor Dir B  |
| 5V pin       | L298N 5V output         | Regulated Power In |

Figure 2: Pin Connections and their labels

## 5.2 Power Architecture

The 12V battery powers the L298N motor supply rail exclusively. The L298N onboard regulator steps this down to a stable 5V which is fed directly to the Arduino 5V pin (bypassing Vin and the internal regulator). The HC-05 module draws 5V and GND from the Arduino. Ceramic and electrolytic capacitors are placed across each motor terminal to suppress back-EMF spikes.

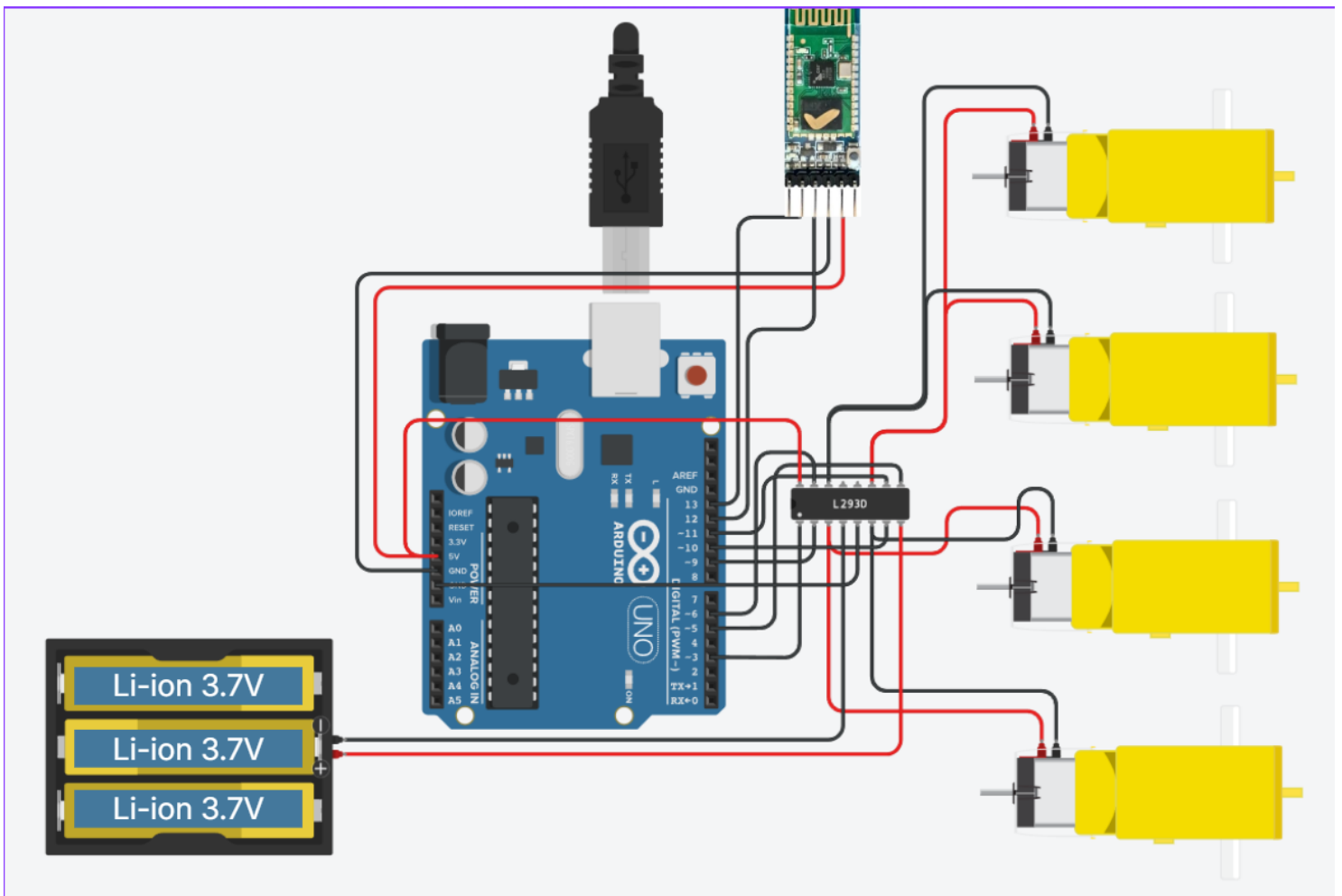


Figure 3: Circuit Diagram – Bluetooth RC Car Wiring

## 6. Working Prototype

The prototype consists of a 4-wheel chassis with two DC gear motors on each side (left and right) wired to the L298N motor driver. The Arduino Uno is mounted centrally on the chassis. The HC-05 Bluetooth module is positioned as far from the L298N as physically possible to minimise RF interference. The 12V battery pack is secured at the rear.

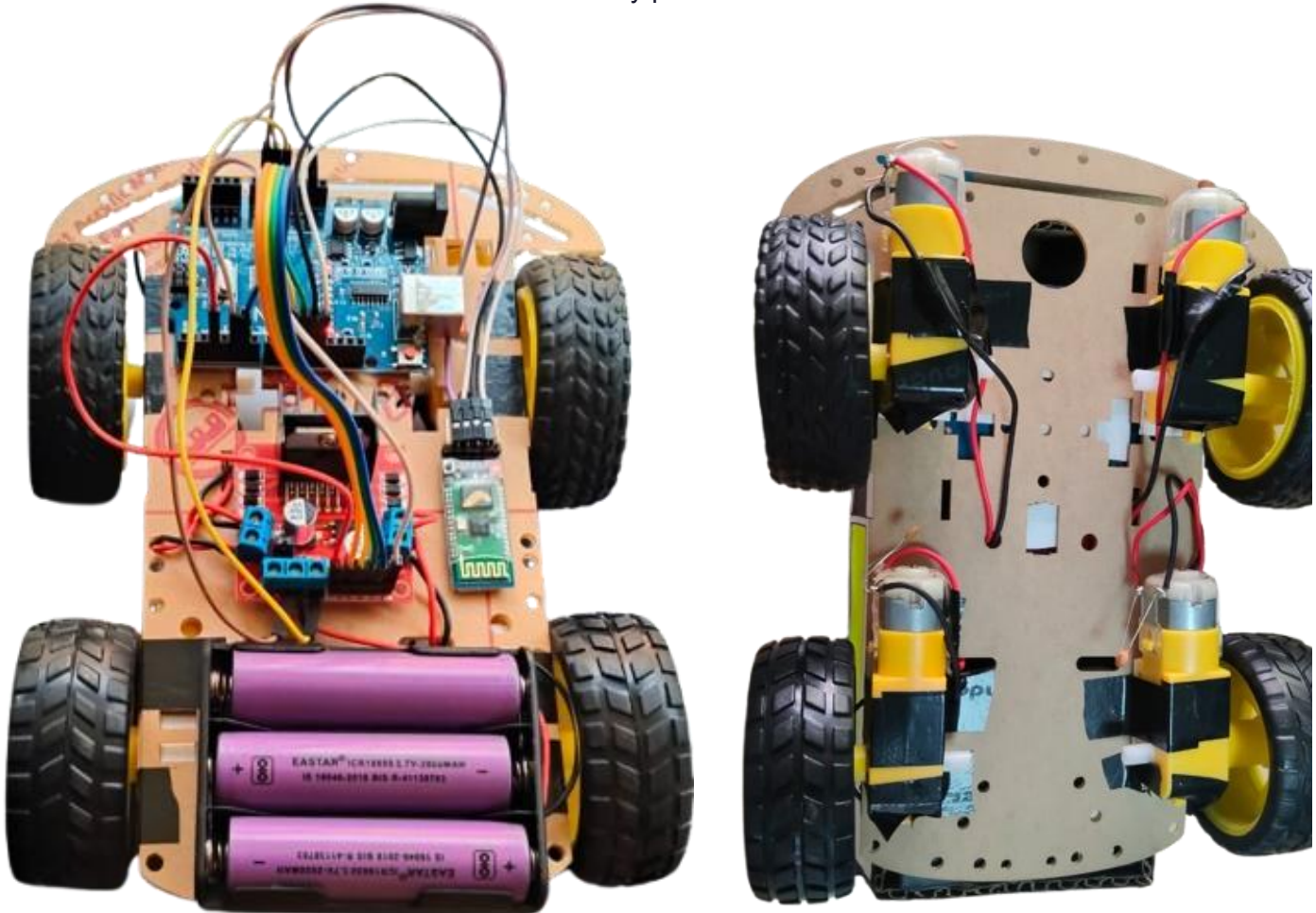


Figure 4: Top view of assembled RC car showing Arduino, L298N, and HC-05 placement



Figure 5: Working Prototype – Side View (insert actual photo here)

## 7. MIT App Inventor Screenshots

The Android application was built entirely in MIT App Inventor. The interface features a Bluetooth device picker, a D-pad style directional control layout, a speed slider ranging from PWM 100 to 180, and a dynamic speed label. The TouchDown and TouchUp events are used to start and stop the car, while the slider continuously updates motor speed in real time.

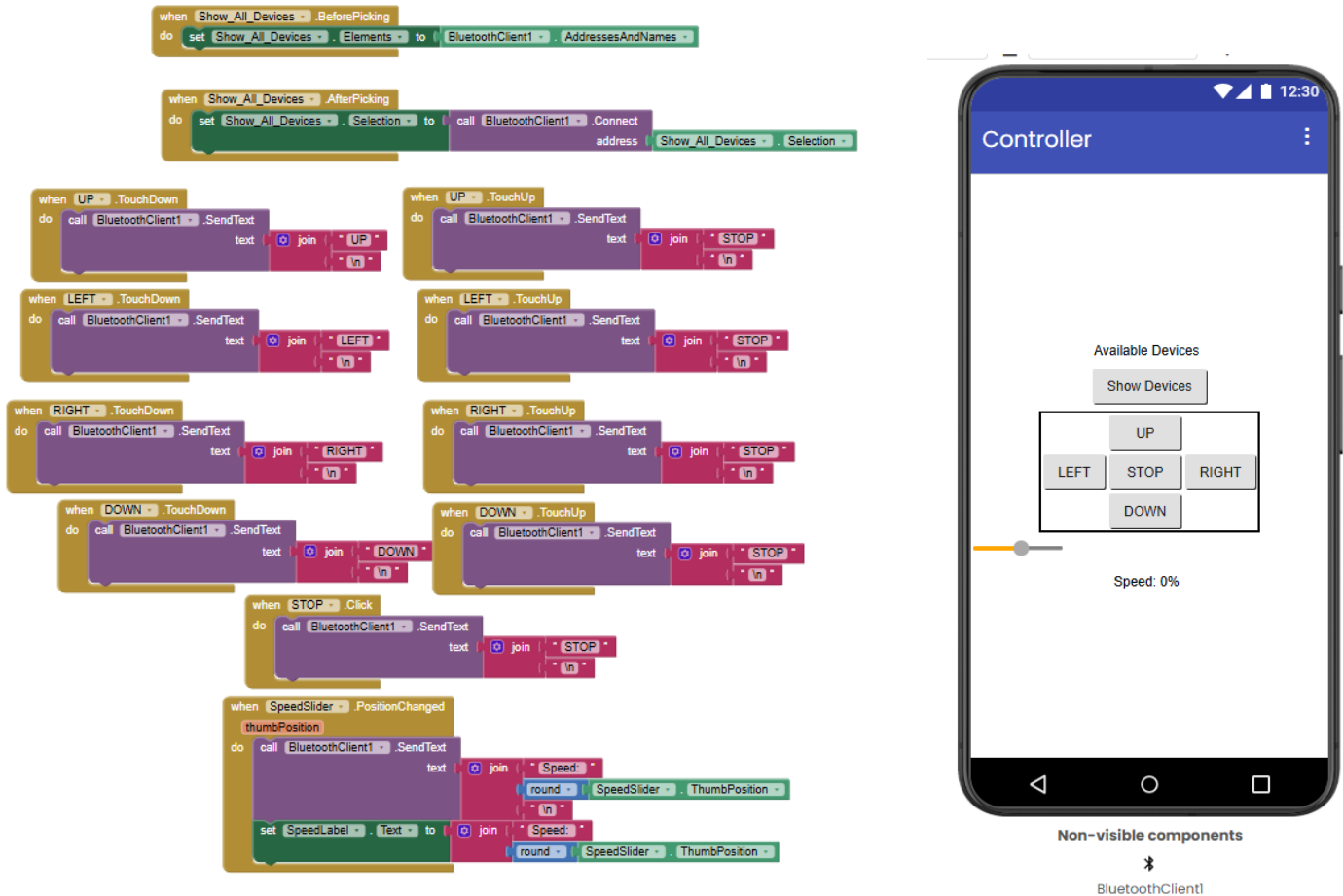


Figure 6,7: MIT App Controller Screen – Bluetooth D-Pad with Speed Slider

### 7.1 App Block Logic Summary

| Event                         | Action / Text Sent                         |
|-------------------------------|--------------------------------------------|
| Show Devices – BeforePicking  | Load paired Bluetooth addresses and names  |
| Show Devices – AfterPicking   | Connect to selected HC-05 device           |
| UP – TouchDown                | SendText: "UP\n"                           |
| UP – TouchUp                  | SendText: "STOP\n"                         |
| LEFT – TouchDown              | SendText: "LEFT\n"                         |
| RIGHT – TouchDown             | SendText: "RIGHT\n"                        |
| DOWN – TouchDown              | SendText: "DOWN\n"                         |
| STOP – Click                  | SendText: "STOP\n"                         |
| SpeedSlider – PositionChanged | SendText: "Speed:<value>\n" + update label |



## 8. Code Flowcharts

### 8.1 Arduino Code Flowchart

The Arduino firmware follows a non-blocking event-driven loop. After initialisation, it continuously polls the Bluetooth serial buffer. A 100 ms timeout window ensures partial commands are handled gracefully. Speed commands are parsed and stored globally; movement commands directly call the corresponding motor function. A watchdog timer auto-stops the car if no command is received for over 300 ms.

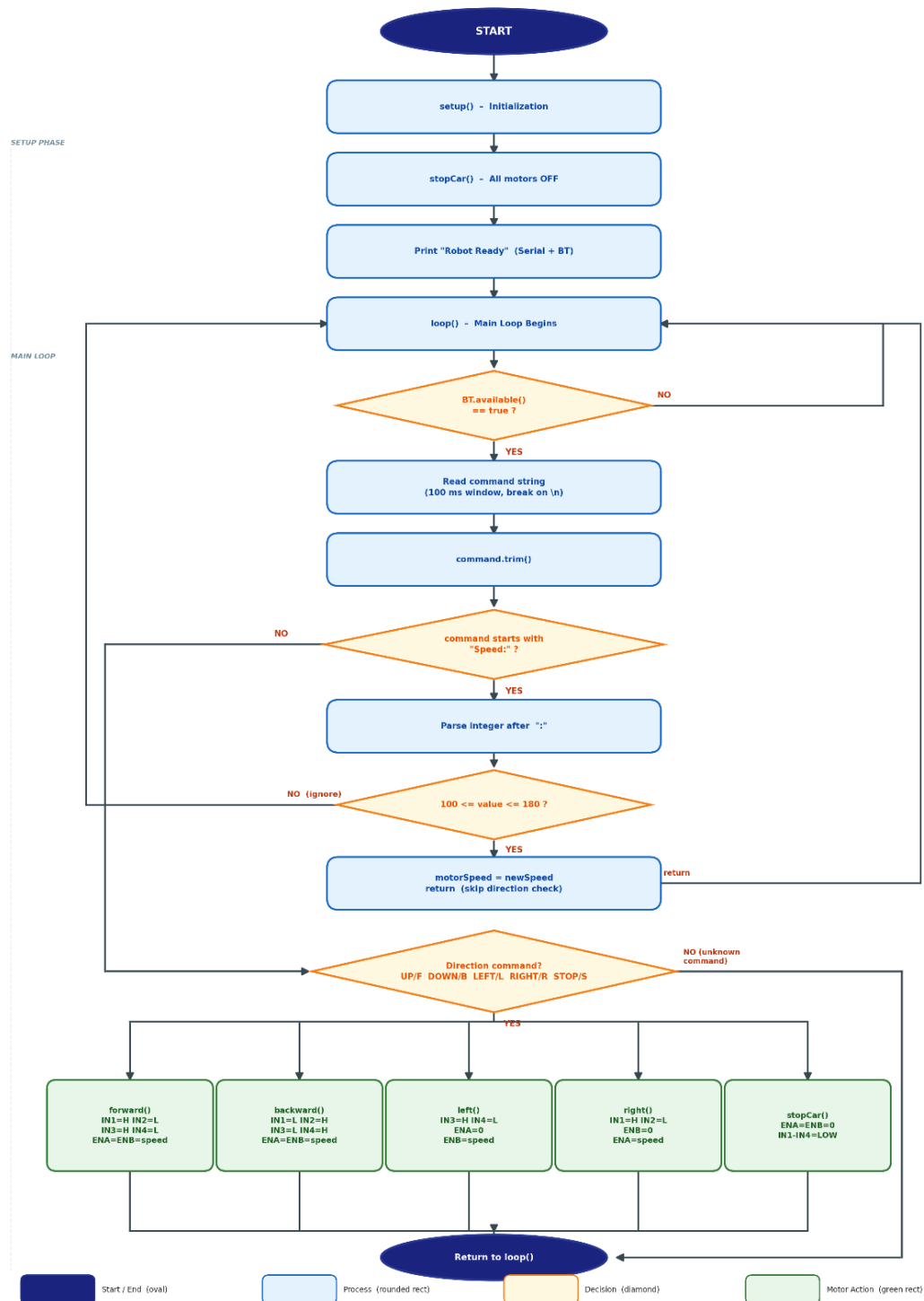


Figure 8: Arduino Code Flowchart – Command Parsing and Motor Control Logic

## 8.2 MIT App Inventor Flowchart

The app follows a simple event-driven model. On launch, the user selects a paired Bluetooth device. Once connected, the D-pad and slider become active. TouchDown events transmit movement commands; TouchUp events transmit STOP. The slider fires PositionChanged continuously, sending speed values in real time.

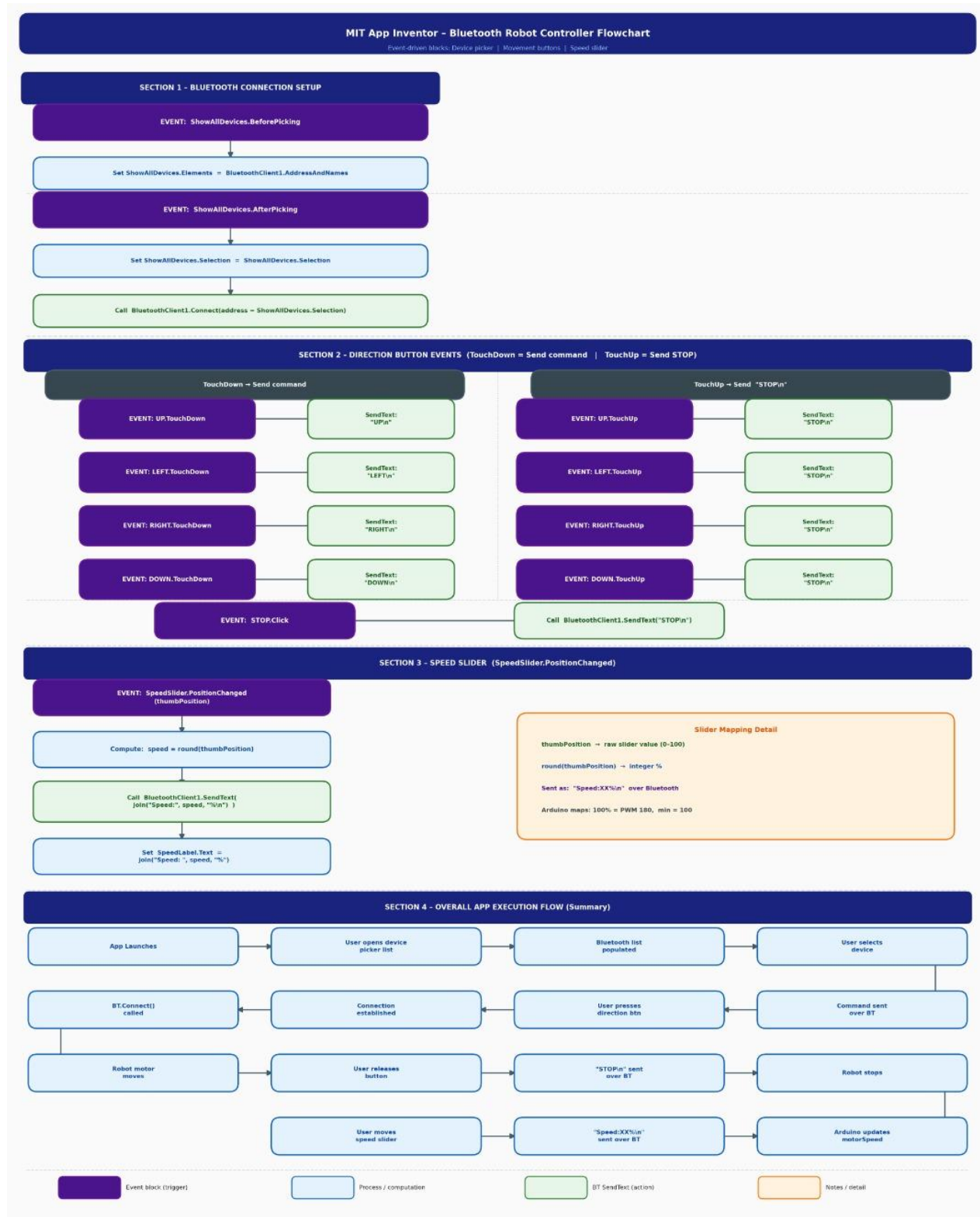


Figure 9: MIT App Inventor Flowchart – Bluetooth Connection and Command Flow



## 9. GitHub Repository

The complete Arduino source code and MIT App Inventor .aia project file are uploaded to the following GitHub repository:

[https://github.com/NishitNfreak/Bluetooth\\_RC\\_CAR.git](https://github.com/NishitNfreak/Bluetooth_RC_CAR.git)

*The repository contains: Arduino sketch (.ino), MIT App Inventor project (.aia), circuit diagram, wiring notes, and this report.*

## 10. References

---

- Arduino Official Documentation – [arduino.cc/en/Reference](http://arduino.cc/en/Reference)
- MIT App Inventor Documentation – [appinventor.mit.edu/explore/ai2/support](http://appinventor.mit.edu/explore/ai2/support)
- L298N Motor Driver Datasheet – STMicroelectronics, Doc ID 1773
- HC-05 Bluetooth Module AT Command Reference – Wavesen Technology
- SoftwareSerial Library Reference – [docs.arduino.cc/learn/built-in-libraries/software-serial](http://docs.arduino.cc/learn/built-in-libraries/software-serial)